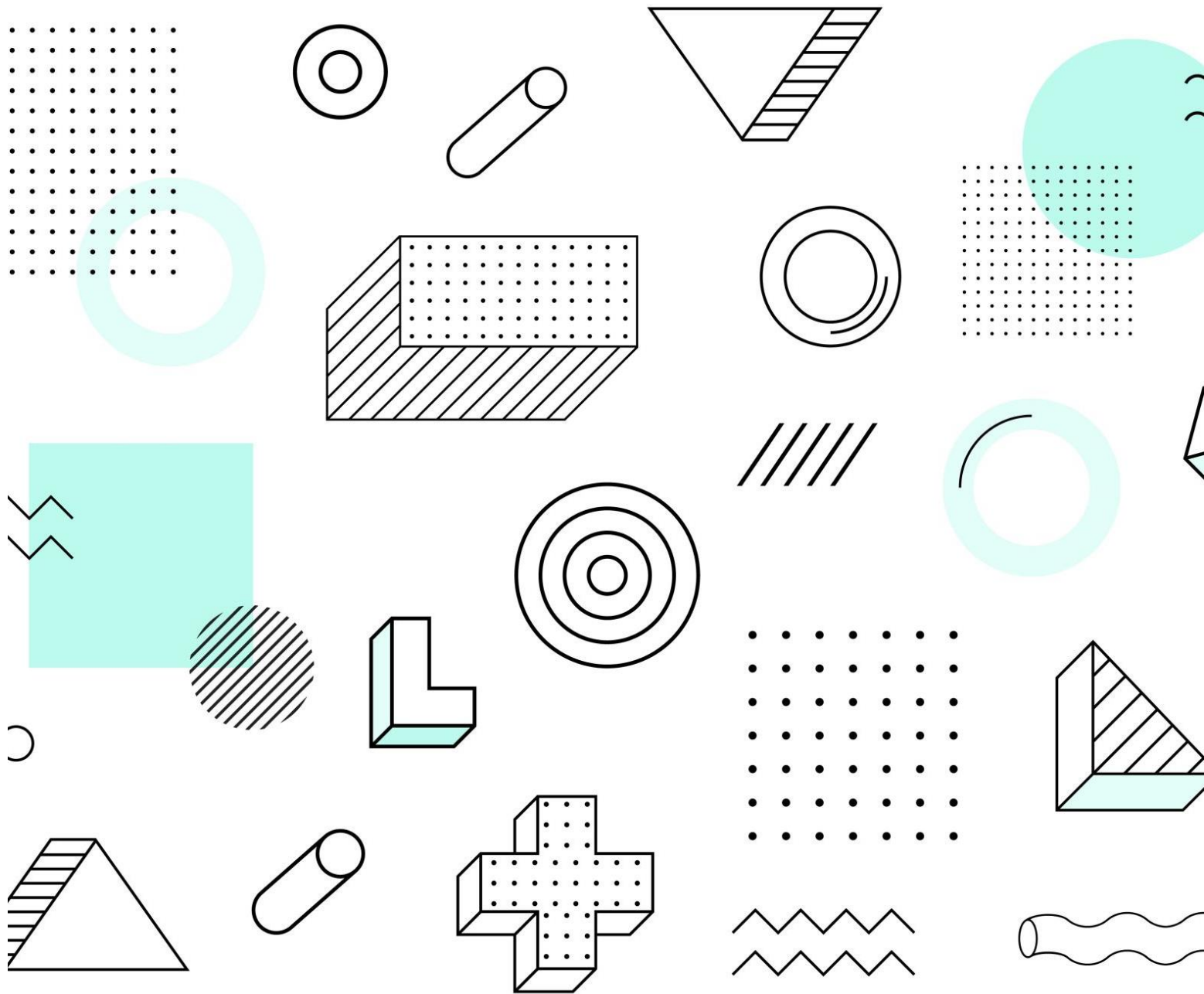


Ciclistas: Miss and Out

Programação Concorrente

Otávio Garcia Capobianco
NUSP: 15482671



Introdução

- Neste EP, implementamos um simulador de ciclistas competindo num velódromo, num modelo “Miss and Out”, sendo cada ciclista representado por uma thread única
- Como cada ciclista age e se move de maneira individual, sem o controle direto de uma entidade central, são necessárias precauções para evitar condições de corrida e deadlocks
- Nesta apresentação, discutiremos o desempenho de tempo e de uso de memória do programa, analisada sob condições variadas de:
 - Tamanho da pista
 - Quantidade de ciclistas
 - Abordagem, relativa a como tratamos condições de corrida, havendo uma forma simples porém ineficiente, e outra mais complexa mas (teoricamente) mais eficiente em tempo gasto

Especificações

- Os experimentos aqui detalhados foram realizados num computador com:
 - Processador **Intel Core i5-5350U** com 2 núcleos físicos e 4 núcleos lógicos
 - 8GB de memória RAM DDR3
- O programa supõe um sistema operacional **Linux**, e a distro utilizada durante a execução dos programas foi o Ubuntu (v24.04.2)
- Todo o código do simulador implementado foi escrito em C, e os gráficos presentes nesta apresentação foram gerados em Python, utilizando Matplotlib

Implementação

- Primeiro, tratemos de como implementamos a abordagem eficiente
- Com o objetivo de evitar a espera desnecessária da abordagem ingênua, criamos uma **matriz de mutexes**, `lock_posicoes`, onde **cada mutex corresponde a uma posição** de pista, de modo que cada ciclista precisa bloquear apenas os mutexes `lock_posicoes[i][j]` relevantes.
- Em qualquer dado momento, **um ciclista trava no máximo 2 mutexes**: um de sua própria posição (ou a posição vertical que está considerando, na ultrapassagem), e outro da posição na sua frente, para onde pretende se mover. Conforme ele considera novas faixas para ultrapassar, destrava os mutexes da faixa anterior
- Para evitar um problema de deadlock tal qual o dos filósofos famintos, o caso em que uma dada faixa está cheia e todos os ciclistas dela vão se mover é considerado antes, e cada ciclista move-se para a frente sem a necessidade de mutexes

Implementação

- Para garantir que um ciclista não ignore uma posição para a qual ele poderia se mover logo após o ciclista da frente se mover (no mesmo turno do relógio), aquele ciclista vai esperar que o da frente acabe suas ações daquele turno. Por conta disso, quando um ciclista acaba o que ia fazer, **envia um cond_signal para sua posição original**. Desse modo, nesta abordagem, precisamos também de uma **matriz de variáveis de condição**, denotada `cond_posicoes`
- Embora o gerenciamento desses semáforos e condições introduza um overhead e gaste mais memória do que a outra abordagem, supomos que o ganho de desempenho de tempo será significativo o bastante para ignorar esses pequenos custos.

Experimentos

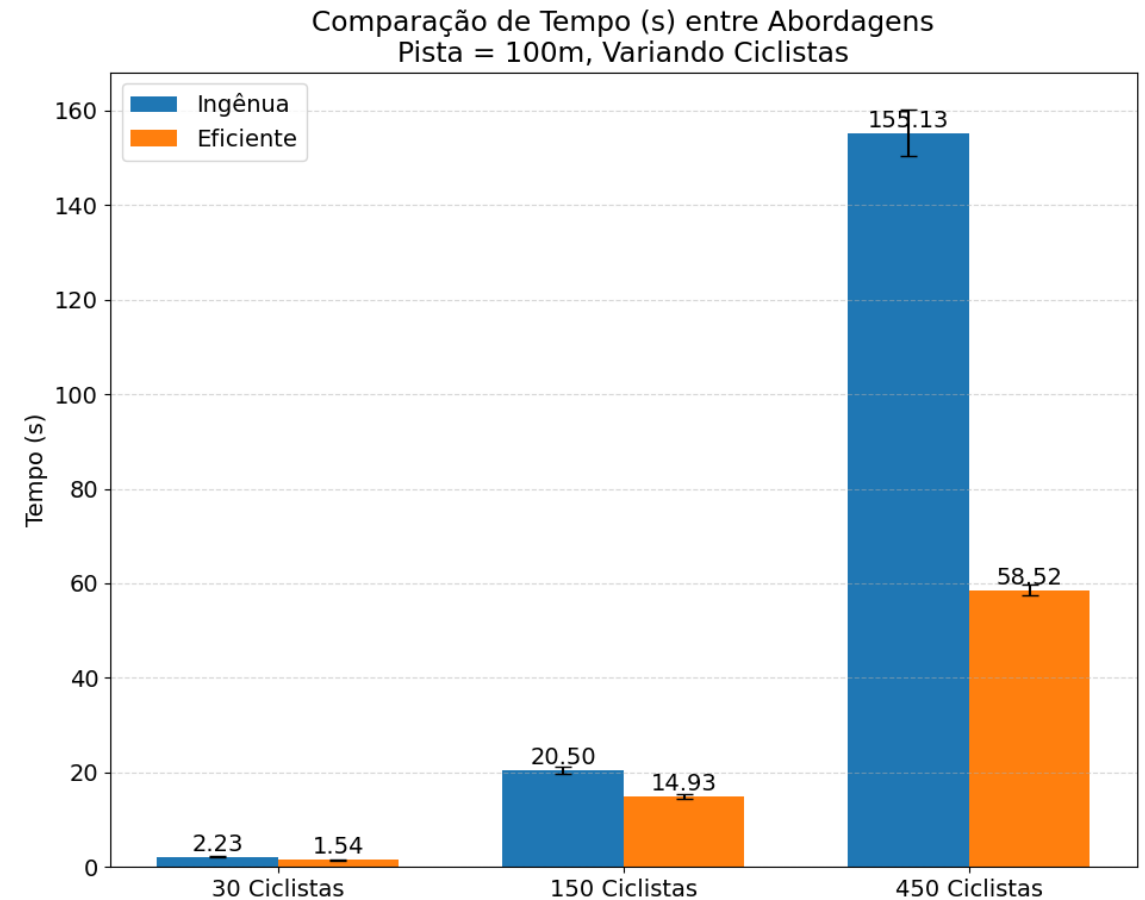
- Para avaliar o desempenho do programa, realizamos testes da seguinte forma:
 - Para um tamanho de pista fixo (100m), variamos a quantidade de ciclistas entre poucos (30), normal (150) e muitos (450)
 - Para uma quantidade de ciclistas fixa (100), variamos o tamanho da pista entre pequena (100m), média (300m) e grande (600m)
- Para cada um dos acima, consideramos as 2 abordagens, totalizando 12 combinações distintas.
- Para cada uma das combinações, rodamos o programa 30 vezes, e os resultados dispostos a seguir incluem a média e intervalo de confiança com nível de confiança 95% resultantes dessas 30 medições
- Vale notar que, para melhorar a coerência dos resultados, apenas o SO e terminais com o próprio EP estavam rodando simultaneamente a essas medições

Experimentos

- Para o tempo gasto, existem duas principais fontes de custo
 - **Quantidade de turnos** (intervalos de 60ms) necessários para acabar, que é impactada tanto pelo **tamanho da pista** (demora mais para ter voltas pares e quebras) quanto pela **quantidade de ciclistas** (são necessárias mais eliminações e quebras)
 - Quanto tempo leva a **execução de cada turno**, que é impactado pela **quantidade de ciclistas** (mais cálculos de movimento e de fim de volta para realizar) e pelo uso da **abordagem ingênua** (mais espera desnecessária)
- Para o uso de memória, temos as causas
 - Representar as matrizes e vetores dependentes do **tamanho da pista**. Aqui, temos também os mutexes e conds extra da **abordagem eficiente**
 - Guardar informações de **cada ciclista**
 - Estruturas de dados que gerenciam **fins de volta, eliminações e quebras**, o que depende de **quantidade de ciclistas** finalizando voltas
- Esperamos que os resultados dos testes realizados concordem com o exposto acima

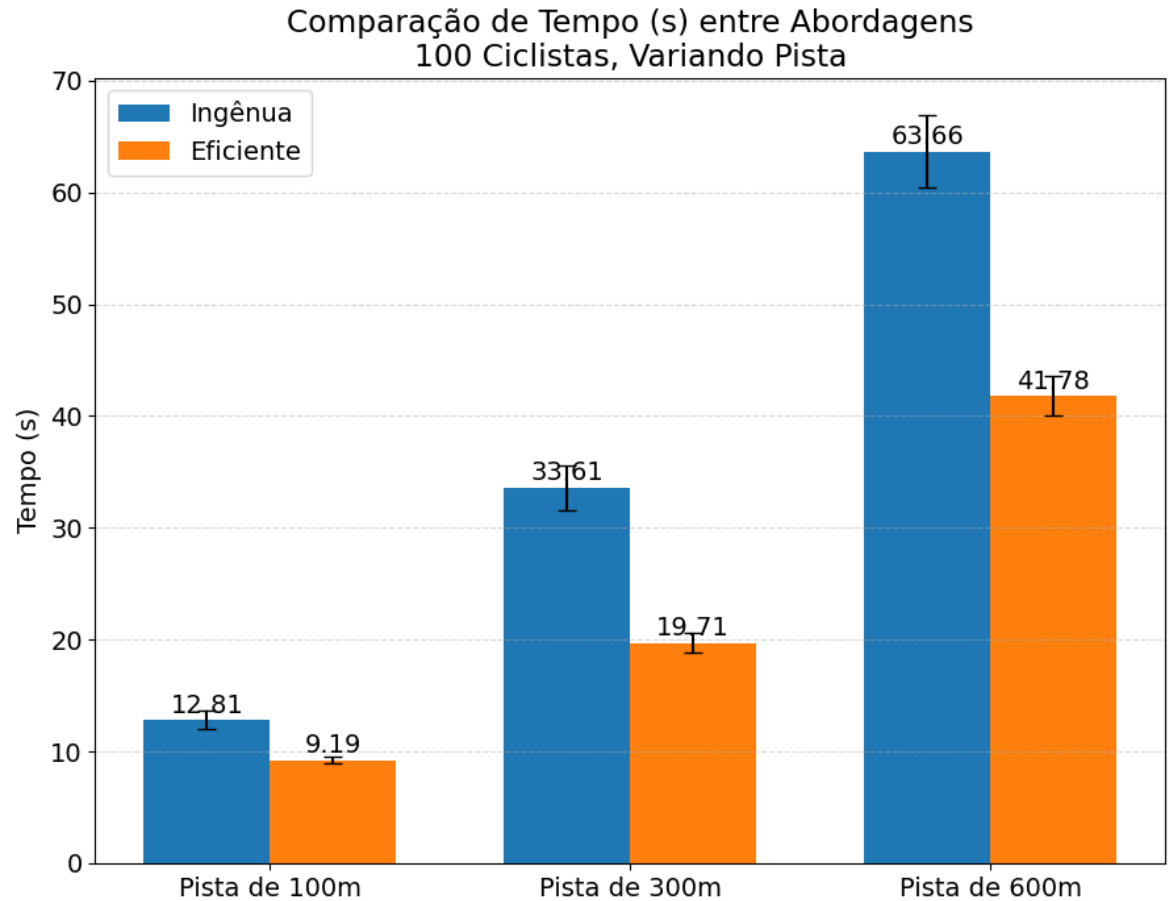
Experimentos – Tempo x Ciclistas

- Aqui, notamos claramente que o tempo aumenta de acordo com o aumento dos ciclistas, e que a abordagem eficiente é melhor que a ingênua, como esperado
- É interessante notar, também, que a **diferença proporcional entre as duas abordagens cresce** com o aumento dos ciclistas, sendo (pela média) a eficiente em torno de 30% mais rápida para 30 e 150 ciclistas, mas 62% para 450
- Isso ocorre pelo aumento de ciclistas em espera desnecessária, que eleva o tempo de cada turno, junto de mais turnos necessários
- Para mais ciclistas, há um maior intervalo de confiança. Entretanto, o crescimento do intervalo parece proporcional ao aumento da média de cada teste, o que é natural



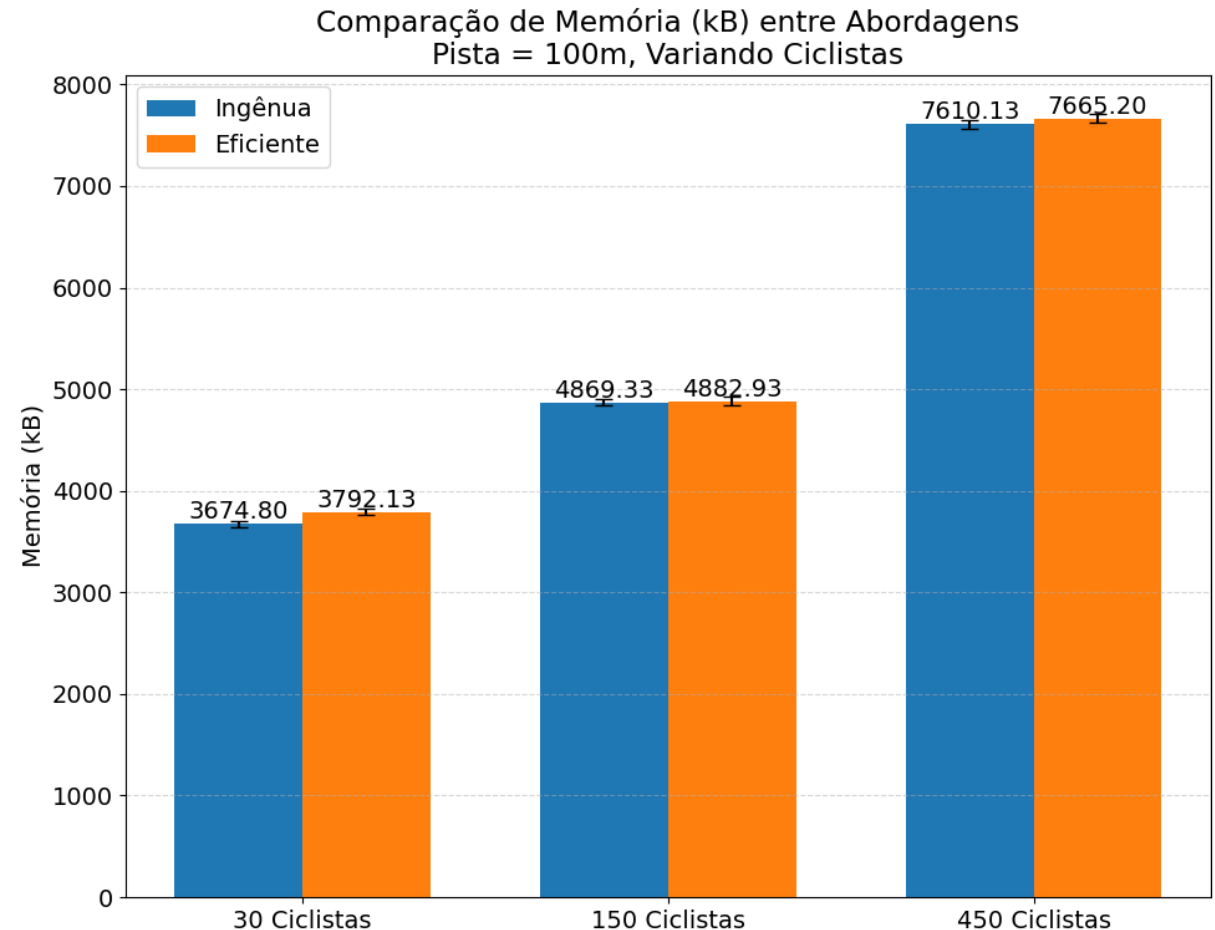
Experimentos – Tempo x Pista

- Novamente, como esperado, quanto maior a pista, mais tempo leva o programa
- As diferenças de tempo entre os tamanhos são apenas pela **quantia de turnos extra** necessária, não pelo tempo de cada turno, o que concorda com o discutido antes
- Diferente do último teste, a **proporção entre as abordagens manteve-se a mesma**. Isso faz sentido, pois o tempo extra da ingênua é dado apenas pela quantidade de ciclistas esperando desnecessariamente. Logo, sem aumentar os ciclistas, esse tempo a mais é proporcionalmente o mesmo



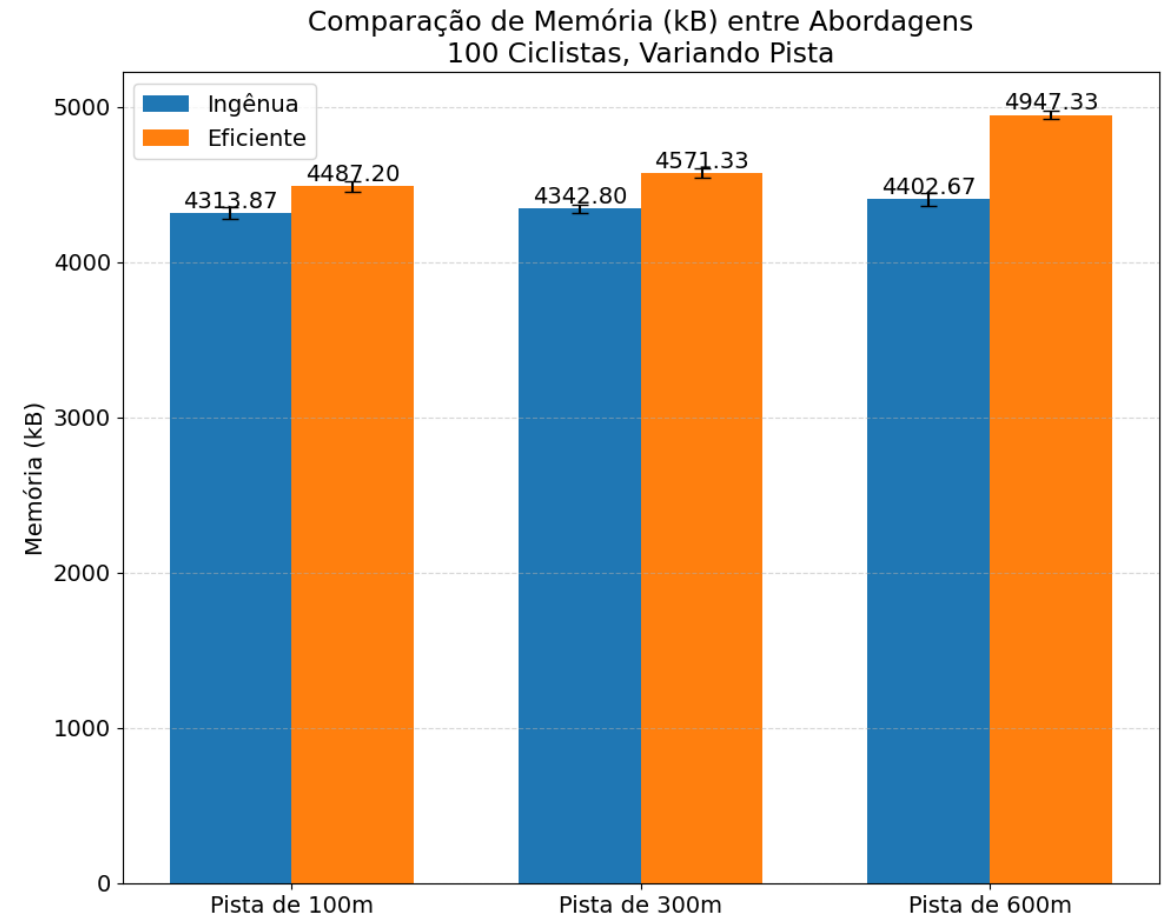
Experimentos – Memória x Ciclistas

- O aumento de ciclistas eleva a memória utilizada, principalmente pela última razão proposta anteriormente, relativa ao uso de **estruturas que gerenciam voltas** finalizadas e eliminações
- Como esperado, a **abordagem eficiente** usa em geral um pouco de **memória a mais**, principalmente pela citada necessidade de criar matrizes de mutexes e de conds



Experimentos – Memória x Pista

- Aqui, embora a memória gasta aumente com o tamanho da pista, o aumento em si é bem pequeno. Isso indica que há uma **correlação mais forte entre ciclistas e memória** do que tamanho de pista e memória
- A razão para tal é que as já mencionadas estruturas de gerência de voltas podem facilmente se **acumular** quando há muitos ciclistas concluindo voltas em um curto intervalo de tempo
- O tamanho da pista acaba por impactar apenas algumas matrizes e vetores de tamanho fixo, que dependem dele



Conclusão

- Prever com precisão o comportamento de um sistema com tantas variáveis móveis e “independentes”, como os ciclistas, pode ser complicado e levar a resultados não tão óbvios
- De modo geral, no entanto, os resultados aqui dispostos **seguiram o que era esperado**
- O impacto de adicionar mais ciclistas, especificamente, é interessante, pois torna a execução do programa significativamente mais custosa, para pequenas variações na quantidade. Para o limite de ciclistas que espera-se que o programa admita (12500), por exemplo, seria inviável rodá-lo 30 vezes para realizar os testes
- Esse impacto, também, revela a importância de um **bom gerenciamento de múltiplas threads** rodando simultaneamente, como visto na diferença entre a abordagem eficiente e ingênua. Uma maior quantidade de cores também poderia apresentar uma redução significativa da perda de performance que esses vários ciclistas trazem. Por estas razões, é importante ter uma máquina adequada e um planejamento bem estruturado ao lidar com problemas de programação concorrente